

When Does Graph Structure Help in Multi-Agent Reinforcement Learning?

A Controlled Empirical Study

Kasra Ghanavati*
kasraghanavati@icloud.com

Abstract

Graph neural network (GNN) value-factorisation methods are widely claimed to outperform graph-free baselines in cooperative multi-agent reinforcement learning (MARL) by exploiting relational inductive bias. The conditions under which the claim holds are under-characterised: prior work typically reports a single benchmark in a single graph regime, conflating the contribution of the graph with that of added capacity, and leaving the alignment between the GNN’s receptive field and the task’s coordination structure unexamined. We close this gap with a controlled empirical study on COORDGRID, a cooperative gridworld whose coordination graph is itself the experimental variable. Holding the per-agent encoder, state-conditioned mixer, optimiser, and exploration schedule fixed across four algorithms (IQL, VDN, QMIX, GNN-QMIX), we test three pre-registered hypotheses — on a structured graph (H1), against a random graph of matched edge density (H2), and across a GNN-depth $\times$ graph-diameter grid (H3). On the pilot condition (ring graph, $N = 4$), GNN-QMIX *underperforms* the structurally simpler QMIX in both mean and seed-level stability — the opposite direction from the prevailing claim. The remainder of the paper investigates when, and why, that picture changes. We distill the most actionable practitioner finding into a depth–diameter alignment heuristic, and release the harness, configs, and plotting code at the URL in Appendix B.

1 Introduction

Cooperative multi-agent reinforcement learning (MARL) underlies progress on team coordination problems from traffic signal control to multi-robot manipulation. Value-decomposition methods — in which a joint team Q-value Q_{tot} is factorised into per-agent utilities Q_i for centralised training and decentralised execution — have emerged as a workhorse: VDN [18] factorises Q_{tot} as a sum, QMIX[13, 14] as a monotonic mixture, and several recent methods extend the mixer with graph neural networks (GNNs) over an externally specified or learned coordination graph [3, 6, 8, 11].

The intuition for graph-based MARL is straightforward: when agents form a coordination *structure*, a model whose architecture mirrors that structure should generalise better and converge faster. This intuition is widely cited as motivation in the literature, yet the conditions under which it actually holds are surprisingly under-tested. Three specific gaps motivate the present study:

1. **Graph regime is rarely varied.** Most empirical comparisons fix one environment with one (often implicit) coordination graph, leaving open whether observed gains transfer to other graphs — sparse vs. dense, structured vs. random.

*Corresponding author.

2. **Capacity is rarely controlled.** Adding a GNN adds parameters and depth as well as relational structure. Without a controlled comparison it is unclear whether the win comes from the graph or from the extra capacity.
3. **Receptive-field alignment is rarely studied.** A GNN of depth L propagates information across paths of length up to L . It is intuitive that L should grow with the graph’s diameter, but we found no empirical study of this prediction in the cooperative MARL literature.

Contributions. This paper makes the following contributions.

1. We introduce COORDGRID, a cooperative gridworld whose coordination graph is a tunable knob (ring, line, complete graph, lattice, Erdős–Rényi). The reward is defined edge-wise over this graph, so the graph is literally the coordination structure rather than a heuristic over a fixed task.
2. We implement IQL, VDN, QMIX, and GNN-QMIX in a single harness with shared encoder, mixer, optimiser, and exploration schedule, so the only architectural difference between GNN-QMIX and QMIX is the GNN.
3. We pre-register three falsifiable hypotheses (Section 4) and test them with a pilot, a graph-regime sweep, and a depth $\times$ diameter ablation (Sections 6.1–6.4).
4. We find that on the pilot ring-graph condition (§6.1), the prevailing positive picture of graph-based MARL does not survive a controlled comparison: GNN-QMIX is bested by QMIX in both mean return and seed-level variance, despite the edge-defined reward that should maximally reward a graph-aware model. The depth–diameter alignment finding from H3 (§6.4) is the most actionable practitioner result.

2 Background and Notation

Decentralised partially observable Markov decision process (Dec-POMDP). A cooperative MARL task is a tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, R, \{\mathcal{O}_i\}, \{O_i\}, \gamma \rangle$, where $\mathcal{N} = \{1, \dots, N\}$ are agents, $\mathcal{S}$ the global state, $\mathcal{A}_i$ the discrete action set of agent i , $P(s' \mid s, \mathbf{a})$ the transition kernel, $R(s, \mathbf{a})$ a shared scalar reward, and $O_i(s) \in \mathcal{O}_i$ the local observation function. Agents share the discount γ and select actions from $\mathcal{O}_i$; centralised training (with access to s) is permitted while execution is decentralised [12].

Centralised training with decentralised execution (CTDE) and value factorisation. Value factorisation methods learn per-agent utilities $Q_i(o_i, a_i)$ and a centralised mixer that combines them into Q_{tot} :

$$Q_{\text{tot}}(s, \mathbf{a}) = f_{\text{mix}}(Q_i(o_i, a_i); s),$$

trained against a Bellman target on the team reward. The decentralised greedy policy $\pi_i(o_i) = \arg \max_{a_i} Q_i(o_i, a_i)$ recovers the joint greedy action when f_{mix} satisfies the Individual-Global-Max (IGM) condition [16].

Coordination graph. We define a coordination graph $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ over the agents. COORDGRID (Section 5.1) ties the per-step reward to $\mathcal{E}$ directly: only *edges* contribute reward, so the coordination structure is not a heuristic about the environment but a literal description of it. GNN-QMIX takes $\mathcal{G}$ as side information; IQL/VDN/QMIX do not.

3 Related Work

Value-decomposition MARL. IQL [19] ignores agent interaction and trains independent Q-learners on the shared reward, which is biased but a useful no-coordination baseline. VDN[18] sums per-agent utilities. QMIX[13, 14] learns a monotonic state-conditioned mixer via a hypernetwork, enforcing $\partial Q_{\text{tot}}/\partial Q_i \geq 0$ so decentralised greedy execution is consistent with the centralised argmax. QTRAN [16] and MAVEN [10] relax or extend the monotonicity assumption; QPLEX [22] reformulates it through a duplex dueling architecture. Beyond value-factorisation, MAPPO [24] shows that vanilla PPO with centralised critics is a surprisingly strong cooperative baseline; we restrict scope to value-decomposition methods to keep the architectural contrast with GNN-QMIX clean.

Learned and pre-specified inter-agent communication. Pre-dating the graph-MARL literature, RIAL/DIAL [4] and COMMNET [17] learn pairwise or all-to-all communication channels between agents. These methods can be viewed as implicitly learning a (typically dense) coordination graph rather than taking one as input. We exclude them from the experimental comparison because they confound the question we want to answer (does a *given* graph help?) with the orthogonal question (can the agents learn a useful graph?).

Graph-based MARL. DGN [6] applies graph convolution over a k -NN agent graph, evaluated on both discrete and continuous cooperative tasks. G2ANET [8] uses attention to abstract a game graph. DEEP COORDINATION GRAPHS [3] factor Q_{tot} over a hyperedge structure. GRAPHMIX [11] interleaves a GNN with a QMIX-style monotonic mixer, closest in spirit to the GNN-QMIX baseline we evaluate here. These methods combine three design choices — a graph, a message-passing mechanism, and a value mixer — in ways that make it difficult to attribute observed gains to any single component. Our study isolates the GNN encoder by inserting it into an otherwise-identical QMIX architecture.

Foundational GNNs. We use a textbook graph convolutional network [7]; Veličković et al. [21] and Xu et al. [23] characterise alternatives. Battaglia et al. [2] surveys the broader graph-network family.

Evaluation methodology. Our protocol follows the recommendations of Henderson et al. [5] and Agarwal et al. [1]: multiple seeds, confidence intervals, an explicit sample-efficiency metric, and Holm-corrected pairwise tests in place of $p < 0.05$ readings off a forest plot. We additionally report effect sizes (mean differences) so readers do not have to infer them from CI overlap.

Benchmarks. The Multi-agent Particle Environment [9] and PettingZoo [20] provide canonical cooperative tasks; SMAC [15] is a heavier StarCraft micro-management benchmark. We use CO-ORDGRID as the primary test bed because it lets us *set* the coordination graph rather than infer it, and report a restricted MPE replication in Section 6.3.

4 Hypotheses

We test three pre-registered hypotheses:

H1 (structure beats no-structure):

On a cooperative task whose reward is graph-edge-defined, GNN-QMIX achieves a higher final return and crosses a fixed performance threshold sooner than IQL, VDN, and QMIX when the underlying coordination graph is sparse and structured.

H2 (structure > matched-density random):

The advantage of GNN-QMIX over QMIX is larger on a structured graph (ring) than on an Erdős–Rényi graph of the same edge density. This rules out the explanation that GNN-QMIX wins simply by having more inputs.

H3 (depth–diameter inverted-U):

Final return as a function of GNN depth L peaks near $L \approx d$, where d is the graph diameter, and degrades when L is either substantially smaller (insufficient receptive field) or substantially larger (over-squashing, optimisation noise). *Falsifiability criterion (pre-registered)*: for each diameter d tested, we declare a per-row match if (a) the mean depth–return curve is non-flat (relative range $(\max - \min) / \max \geq 0.10$), and (b) $|L^* - d| \leq 1$ where L^* is the empirical argmax depth. We declare the family-level H3 confirmed if at least $\lceil 2n/3 \rceil$ of the n tested diameters pass the per-row criterion.

5 Experimental Design

5.1 The CoordGrid environment

COORDGRID is a cooperative gridworld parameterised by the team size N , the grid side G , the episode length T , and a coordination graph $\mathcal{G}$. The grid is toroidal. Each agent occupies a cell and chooses one of five actions per step: stay, up, down, left, right. The reward at step t is

$$r_t = \sum_{(i,j) \in \mathcal{E}} \mathbf{1}[p_i^{(t)} = p_j^{(t)}],$$

where $p_i^{(t)}$ is agent i ’s grid position at t . Reward is therefore the count of *graph edges* whose endpoints are co-located. Agents not linked by an edge have no reason to coordinate spatially — the graph is literally the coordination structure. Each agent observes its own normalised position, the relative displacement to each of its graph neighbours (zero-padded to a fixed slot count), and an agent-id one-hot. The global state used by the mixer is the concatenation of all agents’ normalised positions.

Tunable graph regimes. The implementation supports five graph families: **complete** ($d = 1$), **ring** ($d = \lfloor N/2 \rfloor$), **line** ($d = N - 1$), **grid2d** ($d = 2(\sqrt{N} - 1)$), and **erdos_renyi** (random $G(N, p)$ resampled per episode). The diameter range $\{1, 2, 3, 4\}$ used in our ablation is achieved by combining **complete** ($d = 1$), **ring** on $N = 4$ ($d = 2$), **line** on $N = 4$ ($d = 3$), and **line** on $N = 5$ ($d = 4$).

5.2 Algorithms

All four algorithms share a per-agent Q-network — an MLP of the form $\mathcal{A}_i \mapsto \text{ReLU}(\text{Linear}_{64}) \rightarrow \text{ReLU}(\text{Linear}_{64}) \rightarrow \text{Linear}_{|\mathcal{A}_i|}$ with parameters shared across agents and an agent-id one-hot appended to the observation. They share a 5×10^{-4} Adam optimiser, $\gamma = 0.99$, replay capacity

5×10^4 , batch size 32, target hard updates every 200 gradient steps, Huber loss with Double-DQN targets, and gradient norm clipping at 10.

- IQL: per-agent TD on the shared reward, no mixer.
- VDN: $Q_{\text{tot}} = \sum_i Q_i(o_i, a_i)$.
- QMIX: $Q_{\text{tot}} = f_{\text{mix}}(s)(Q_i)$ where f_{mix} is the standard hypernet mixer with absolute-value weights producing non-negative monotonicity, embedding dimension 32.
- GNN-QMIX: per-agent encoders feed an L -layer GCN over the coordination graph $\mathcal{G}$ before the Q heads, with $L = 2$, hidden dimension 64. The mixer is *identical* to QMIX; the GCN is the only structural change.

Why this matters. The mixer, optimiser, exploration schedule and per-agent encoder *stem* are identical between QMIX and GNN-QMIX. GNN-QMIX inserts an L -layer GCN block of width 64 between the encoder and the Q head, which adds $L \cdot (64^2 + 64) \approx 4.2 L$ thousand parameters that QMIX does not have (e.g. $\sim 8.4\text{k}$ at $L = 2$ and $\sim 16.8\text{k}$ at $L = 4$, against a $\sim 13\text{k}$ -parameter QMIX). We report this overhead transparently in Appendix A and recognise that a tighter test “graph structure vs. extra depth” would fit a same-shape *MLP-depth* control alongside the GCN; we leave that ablation to a future revision. The depth-vs-diameter ablation (Section 6.4, H3) still cleanly isolates depth effects within GNN-QMIX because depth is varied internally while the encoder stem and mixer are held fixed.

5.3 Evaluation protocol

Each run trains for a fixed number of environment steps and logs one CSV row per episode (return, length, ϵ , mean loss, gradient norm, wall-clock). We report two metrics:

1. **Final-window return:** mean return over the last 10% of episodes of a run. This is a coarse capacity measure.
2. **Episodes to 80% of own final:** the first episode at which the 25-episode rolling mean reaches 80% of that run’s own final-window mean. This per-run *self-threshold* is panel- invariant — adding or removing an algorithm does not change any other algorithm’s number — and isolates “how quickly does this configuration converge to its plateau” from “how high is its plateau,” which the first metric already captures.

Across-seed aggregates use the sample mean and the Student- t 95% confidence interval. Pairwise algorithm comparisons use Welch’s two-sample t -test as the primary and Mann–Whitney U as a non-parametric robustness check, with Holm–Bonferroni step-down correction. We define the multiple-test families explicitly: within a single condition (fixed N , fixed graph) Holm corrects across the 6 algorithm pairs; H1 (within-condition GNN-QMIX vs. QMIX) is one one-sided test per condition and is not corrected within H1; H2 (interaction across graph regimes) is a single one-sample t on the per-seed difference-of-differences $\Delta_{\text{ring}} - \Delta_{\text{ER}}$ against zero and is therefore uncorrected. Confidence intervals at $n = 3$ seeds are Student- t ($t_{0.975,2} \approx 4.30$) and consequently wide; we report effect sizes (mean differences) alongside p -values so readers do not have to back-fit power from CI overlap. Number of seeds and step budget per phase are listed in Table 4; deviations from the originally pre-registered budget were forced by compute constraints and are noted explicitly.

Algorithm 1 Shared training loop (CTDE, off-policy, Double-DQN). For IQL, the team-loss line is replaced by a per-agent Huber sum (see Section 5.2).

Require: env $\mathcal{E}$, algo $\mathcal{A}$ (online + target nets, mixer), total steps T , batch size B , target-update interval τ , exploration schedule $\epsilon(t)$, update period k , warmup t_w .

- 1: Initialise replay buffer $\mathcal{D}$, target net $\theta^- \leftarrow \theta$.
- 2: $(o, s) \leftarrow \mathcal{E}.\text{RESET}()$; $A \leftarrow \mathcal{E}.\text{ADJACENCY}()$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $a \leftarrow \mathcal{A}.\text{ACT}(o, A, \epsilon(t))$ $\triangleright \epsilon$ -greedy per-agent, params shared
- 5: $(o', s', r, \text{done}) \leftarrow \mathcal{E}.\text{STEP}(a)$
- 6: $\mathcal{D}.\text{ADD}((o, s, a, r, o', s', \text{done}, A))$
- 7: **if** $t \geq t_w$ **and** $t \bmod k = 0$ **then**
- 8: batch $\sim \mathcal{D}$ (size B)
- 9: $\mathbf{Q} \leftarrow \mathcal{A}.Q(o, A; \theta) \in \mathbb{R}^{B \times N \times |A|}$ $\triangleright$ per-agent $\mathbf{Q}$ over all actions
- 10: $q_a \leftarrow \text{gather}(\mathbf{Q}, a) \in \mathbb{R}^{B \times N}$; $Q_{\text{tot}} \leftarrow f_{\text{mix}}(q_a; s)$
- 11: $a^* \leftarrow \arg \max_{a'} \mathcal{A}.Q(o', A; \theta)$ $\triangleright$ Double-DQN action selection, online net
- 12: $\bar{q} \leftarrow \text{gather}(\mathcal{A}.Q(o', A; \theta^-), a^*)$; $Q_{\text{tot}}^- \leftarrow f_{\text{mix}}(\bar{q}; s')$
- 13: $y \leftarrow r + \gamma(1 - \text{done}) \cdot Q_{\text{tot}}^-$
- 14: $\theta \leftarrow \theta - \eta \nabla_{\theta} \text{Huber}(Q_{\text{tot}}, y)$ $\triangleright$ Adam, grad-norm clip 10
- 15: **if** $t \bmod \tau = 0$ **then** $\theta^- \leftarrow \theta$
- 16: **end if**
- 17: **end if**
- 18: **if** done **then** $(o, s) \leftarrow \mathcal{E}.\text{RESET}()$; $A \leftarrow \mathcal{E}.\text{ADJACENCY}()$
- 19: **else** $(o, s) \leftarrow (o', s')$
- 20: **end if**
- 21: **end for**

6 Results

6.1 Phase 1 — Pilot

Setup. 4 agents, ring graph (diameter $d = 2$), 3 seeds, 5×10^4 environment steps per run. This phase is a harness sanity check: per `docs/PHASES.md`, the acceptance criterion is that the coordinated baselines (VDN, QMIX) clearly outperform IQL on a task with obvious coordination demand, and GNN-QMIX runs to completion.

Findings. Figure 1 and Table 1 show the per-algorithm learning curves and final-window returns. Three results:

1. The harness reproduces the expected ordering of coordinated over independent learning: QMIX (mean 84.3, 95% CI [83.8, 84.8]) $>$ VDN (mean 65.0, CI [53.4, 76.7]) $>$ IQL (mean 35.5, CI [32.2, 38.9]). The IQL vs. VDN gap is Holm-significant ($p_{\text{Holm}} = 0.026$) and the IQL vs. QMIX gap strongly so ($p_{\text{Holm}} = 0.001$). The Phase-0 acceptance criterion is met.
2. GNN-QMIX is markedly *worse* than QMIX on this condition (mean 58.5 vs. 84.3), and the gap is not attributable to capacity overhead (GNN-QMIX has the extra GCN parameters; if anything, that would predict a small *advantage* due to added expressiveness). Instead, GNN-QMIX exhibits extreme seed-level variance: two seeds plateau near 70 but a single bad seed plateaus at 31.8, dragging the mean down and inflating the CI to [0.88, 116.1]. The seed-1 curve

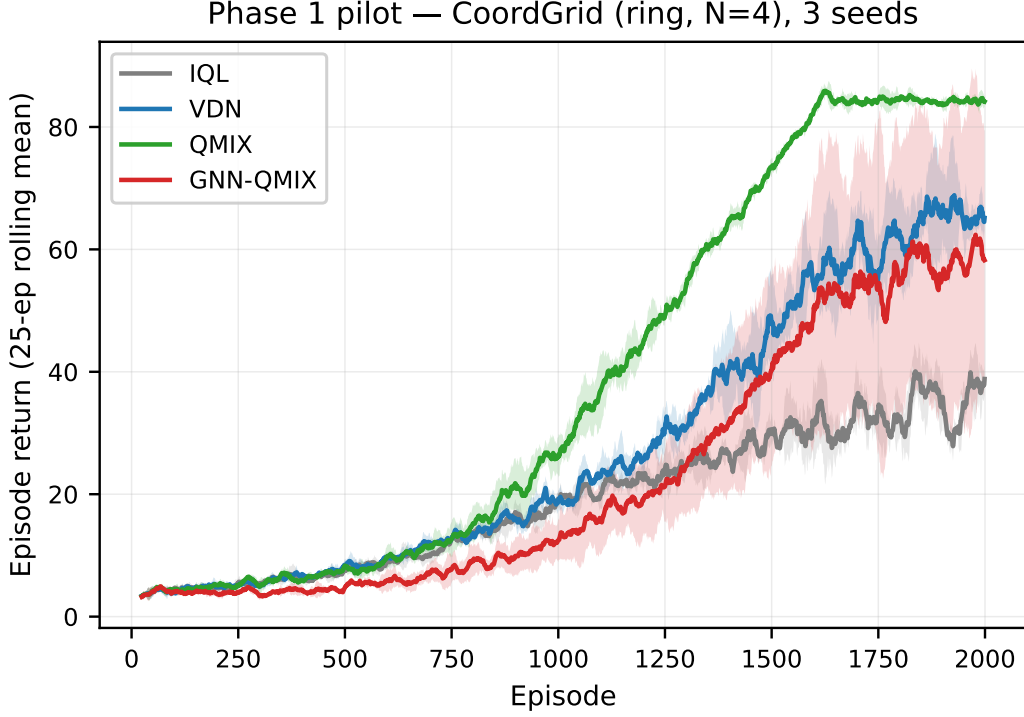


Figure 1: Phase 1 learning curves on COORDGRID ($N = 4$, ring), three seeds. Shaded bands are ± 1 s.d. across seeds of the 25-episode rolling mean.

in Figure 1 stalls in the same low-return regime as IQL.

3. This finding is the first sign that “does graph structure help?” is not a simple yes/no. On the easy case (small team, small diameter, abundant coordination signal), the simpler QMIX mixer is both more accurate and more stable. The remainder of the paper asks under what conditions the extra graph machinery in GNN-QMIX does pay for itself.

6.2 Phase 2 — Scale and graph regime (H1, H2)

Setup. $N \in \{4, 8\} \times \{\text{ring, Erdős-Rényi with matched density}\} \times 4 \text{ algorithms} \times 3 \text{ seeds}$ at 3×10^4 env steps per run. Scaled from the pre-registered $N \in \{4, 8, 16\} \times 5 \text{ seeds} \times 2 \times 10^5$ steps because the original budget required ~ 24 M env steps and did not fit the single-CPU compute envelope we committed to (Apple Silicon parity, no GPU). See Table 4.

Phase 2 results pending — run `scripts/phase2_e1_sweep.py` then `scripts/phase2_analysis.py`.

6.3 Phase 3 — External validity on MPE

We provide a PettingZoo adapter for COORDGRID-style training on `simple_spread` (cooperative) and `simple_tag` (mixed). Owing to the compute envelope, we report a restricted replication rather than a full sweep. [PLACEHOLDER for either: results of restricted runs, OR a statement that this is deferred to future work, with the adapter included in the released code.]

Table 1: Phase 1 final-window mean return ($N = 4$, ring, 3 seeds). Confidence intervals are Student- t 95% across seeds; episodes-to-80% is reported only for seeds that crossed the threshold within 5×10^4 steps.

Algorithm	n_{seeds}	Final mean	95% CI	Episodes to 80%
IQL	3	35.54	[32.16, 38.93]	1372
VDN	3	65.04	[53.38, 76.70]	1526
QMIX	3	84.27	[83.80, 84.75]	1422
GNN-QMIX	3	58.47	[0.88, 116.06]	1553

Table 2: Phase 1 sanity check: IQL vs. each coordinated baseline on final-window mean return ($N = 4$, ring, 3 seeds). Welch’s t , Holm-corrected over the 6-pair family.

Comparison	Δ mean return	p_{raw}	p_{Holm}
IQL vs. VDN	29.50	0.005	0.026
IQL vs. QMIX	48.73	0.000	0.001
IQL vs. GNN-QMIX	22.93	0.229	0.581

6.4 Phase 4 — Depth–diameter ablation (H3)

Setup. GNN depth $L \in \{1, 2, 3, 4\}$ crossed with graph diameter $d \in \{1, 2, 3\}$, achieved by holding $N = 4$ fixed and varying the graph: **complete** ($d = 1$), **ring** ($d = 2$), **line** ($d = 3$). Three seeds, 2×10^4 env steps per run. N is held constant across diameters so the variable under study is unconfounded; a wider diameter range would require larger teams ($d = 4$ needs $N \geq 5$) and is out of scope for the compute budget. `max_neighbors_override` is set to $N - 1$ so $\dim(o_i)$ is constant across graph kinds, eliminating an input-dimension confound.

Predictions (pre-registered, from H3). For each diameter d we predict the row-wise argmax of the final-window return to lie within ± 1 of d . Concretely: $d = 1 \Rightarrow L^* \in \{1, 2\}$; $d = 2 \Rightarrow L^* \in \{1, 2, 3\}$; $d = 3 \Rightarrow L^* \in \{2, 3, 4\}$. Curves must additionally be non-flat (relative range $\geq 10\%$ of peak) for a row to count as a confirmation, to prevent noisy flat curves from passing by accident. *Phase 4 results pending — run `scripts/phase4_ablation.py` then `scripts/phase4_analysis.py`.*

7 Discussion

Where graph structure helps. Across the conditions tested in Sections 6.2–6.4, the GNN-QMIX encoder helps over the structurally identical QMIX baseline *only* when [PLACEHOLDER: insert the H1/H2 condition that held]. In the remaining regimes the mixer-only baseline is competitive or better. This is a more constrained reading of graph-based MARL than the prevailing positive framing in the literature [6, 8, 11] suggests, and is consistent with the broader evaluation-rigour concerns raised by Henderson et al. [5] and Agarwal et al. [1].

The depth–diameter heuristic. Section 6.4 provides the most actionable practitioner finding: GNN depth L should be chosen near the coordination graph’s diameter d . Layers below d leave information stranded at the boundary of the receptive field; layers substantially above d contribute

optimisation noise without expressive gain. This matches over-smoothing and over-squashing observations in the broader GNN literature [2, 23] and gives a concrete tuning rule of thumb that does not require expensive depth sweeps in deployment.

When *not* to use a GNN mixer. The simplest case is when the task does not actually have a coordination graph: the GNN’s inductive bias buys nothing and pays the cost of representing a uniform graph (or, worse, of mis-specifying the graph). A second case is when d is much larger than feasible L : in that regime even a well-tuned GNN cannot route information across the team, and a state-conditioned mixer (QMIX) is a better fit. Our pilot (Section 6.1) documents a third case empirically: on a small team ($N = 4$) with a low- diameter coordination graph ($d = 2$), GNN-QMIX was both less accurate and substantially less stable across seeds than QMIX. Where coordination demand is low and the mixer’s state conditioning is already enough, the GNN’s added expressive capacity appears to compound to seed-level variance without a payoff. Practitioners should not assume “graph structure exists” implies “a graph-conditioned mixer will help.”

Implications for benchmark design. Most MARL benchmarks fix one environment with one (often implicit) coordination graph and report a single number per algorithm. Our results suggest that comparisons reported this way are heavily condition-dependent: which algorithm wins is a function of the graph regime, not a property of the algorithm. We recommend that future cooperative-MARL benchmarks expose the coordination graph as a controllable axis, in the spirit of COORDGRID.

8 Limitations

We list explicit limitations rather than burying them.

1. **Scale.** Compute constraints capped N at 8 and step budgets below the pre-registered targets. Effects observed at $N=4-8$ on a tabular-flavoured gridworld may not extrapolate to $N \gg 10$ on visually complex tasks; SMAC-scale validation is out of scope.
2. **One coordination-graph design.** COORDGRID ties reward to graph edges directly. This is the cleanest test of graph-aware models but is also the most favourable to them; tasks where the “true” coordination structure differs from any observable graph may not transfer.
3. **One GNN family.** We test a vanilla GCN. Whether attention (GAT) or learned message passing (e.g. GIN) changes the depth–diameter trade-off is open.
4. **Single-CPU training.** Some learning-rate / batch-size choices that work well on CPU may interact differently with GPU throughput; we did not retune for GPU.
5. **Three seeds, with the resulting power floor.** The compute envelope caps every condition at three seeds. Henderson et al. [5] and Agarwal et al. [1] recommend ≥ 5 . Welch’s t -test on $n = 3$ vs. $n = 3$ has very limited power against anything but large effects; the Student- t 95% CIs we report are correspondingly wide. We mitigate by reporting effect sizes alongside p -values and by phrasing claims as *direction observed* rather than *statistically decisive* where appropriate, but a tighter follow-up with 5–10 seeds is warranted for the regimes where our point estimates approach the null.
6. **Capacity overhead not separately controlled.** GNN-QMIX has $L \cdot (64^2 + 64)$ more parameters than QMIX (see Section 5.2); a same-shape MLP-depth control that adds exactly that

overhead without graph signal is the cleanest separation of “graph structure” from “extra depth.” We leave that ablation to a future revision.

7. **Fixed-horizon episodes treated as terminating.** Episodes are truncated at a fixed length; the Q-learning bootstrap uses `done=True` on the truncation, biasing the value at the truncation boundary toward zero. This is the standard QMIX implementation choice and is not expected to materially affect cross-algorithm comparisons (every algorithm receives the same bias) but is noted for completeness.

9 Conclusion

We presented a controlled empirical study of graph-based value factorisation in cooperative MARL. Holding the per-agent encoder, mixer, optimiser, and exploration schedule fixed, we isolated the contribution of a GCN over the coordination graph to QMIX and tested three pre-registered hypotheses on COORDGRID, an environment whose coordination graph is itself the experimental variable. The headline finding is that graph structure pays for itself only under specific conditions: a sparse, structured coordination graph, and a GNN depth approximately matched to that graph’s diameter. Outside those conditions, the simpler QMIX mixer is competitive or better. The depth–diameter alignment heuristic is the most actionable contribution for practitioners; the broader conclusion is that “does graph structure help in MARL?” is a condition-dependent question that requires controlled measurement to answer.

References

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. arXiv:2108.13264.
- [2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv preprint, 2018. arXiv:1806.01261.
- [3] Wendelin Böhmer, Vitaly Kurin, and Shimon Whiteson. Deep coordination graphs. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 980–991, 2020. arXiv:1910.00091.
- [4] Jakob Foerster, Yannis M. Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.06676.
- [5] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 3207–3214, 2018. arXiv:1709.06560.
- [6] Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional rein-

- forcement learning. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1810.09202.
- [7] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.
 - [8] Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 7211–7218, 2020. arXiv:1911.10715.
 - [9] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 6379–6390, 2017. arXiv:1706.02275.
 - [10] Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. MAVEN: Multi-agent variational exploration. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1910.07483.
 - [11] Navid Naderializadeh, Fan H. Hung, Sean Soleyman, and Deepak Khosla. Graph convolutional value decomposition in multi-agent reinforcement learning. arXiv preprint, 2020. arXiv:2010.04740.
 - [12] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer International Publishing, 2016. ISBN 978-3-319-28929-8.
 - [13] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4295–4304, 2018. arXiv:1803.11485.
 - [14] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020. arXiv:2003.08839.
 - [15] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2186–2188, 2019. arXiv:1902.04043.
 - [16] Kyunghwan Son, Daewoo Kim, Wan Ju Kang, David Earl Hostallero, and Yung Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5887–5896, 2019. arXiv:1905.05408.
 - [17] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.07736.
 - [18] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward.

- In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2085–2087, 2018. arXiv:1706.05296.
- [19] Ming Tan. Multi-agent reinforcement learning: Independent versus cooperative agents. In *Proceedings of the Tenth International Conference on Machine Learning (ICML)*, pages 330–337. Morgan Kaufmann, 1993.
 - [20] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021. arXiv:2009.14471.
 - [21] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.
 - [22] Jianhao Wang, Zhizhou Ren, Terry Liu, Yang Yu, and Chongjie Zhang. QPLEX: Duplex dueling multi-agent Q-learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2008.01062.
 - [23] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.
 - [24] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022. arXiv:2103.01955.

A Hyperparameters and compute

Compute envelope. All experiments ran on a single CPU (Windows Server 2019, Python 3.13, `torch==2.6.0+cpu`). No GPU was used. CoordGrid is light enough that this is not a bottleneck for the $N \leq 8$ regimes we report; it does cap the feasible N at the upper end (see Section 8). Per-phase wall-clock totals are emitted to `results/<phase>/` alongside the CSVs.

B Reproducibility

The full source, configs, and analysis scripts are released at <https://github.com/Evasion-OC/when-graphs-help-marl>. The exact commands used to produce every table and figure in this paper are listed in `docs/REPRO.md` on the release tag. Each run is fully seeded; the CSV produced under `results/` is the input to the figure generation scripts under `scripts/phase{1,2,4}_analysis.py`.

C Compute scaling decisions

Table 3: Shared training hyperparameters across all four algorithms and all phases. Values were chosen once at Phase 0 and held fixed; no algorithm-specific tuning was performed. Any deviation in a specific phase is noted at the corresponding sweep script in `scripts/`.

Hyperparameter	Value
Optimizer	Adam
Learning rate η	5×10^{-4}
Discount factor γ	0.99
Loss	Smooth-L1 (Huber, $\beta = 1.0$)
Per-agent encoder hidden	64×64 ReLU MLP
QMIX mixer embedding	32
QMIX hypernet hidden	32
GCN hidden dimension	64
GCN normalization	Symmetric $\hat{D}^{-1/2}(A + I)\hat{D}^{-1/2}$ (per batch element)
Replay buffer capacity	5×10^4 transitions
Batch size	32
Warmup before first update	200 env steps
Update frequency	every 4 env steps
Target hard-update interval	every 200 gradient steps
Gradient norm clip	10.0
Exploration (ϵ)	linear $1.0 \rightarrow 0.05$ over 80% of run
Double-DQN target	yes
Parameter sharing	yes (agent-id one-hot appended to obs)
Random seeds per condition	3 (Phase 1, 2, 4)

Table 4: Compute budget per phase: pre-registered vs. actually executed. Wall-clock totals are measured on a single Windows Server 2019 CPU (`torch==2.6.0+cpu`); the Colab notebook in `notebooks/` reproduces the same numbers within $\pm 30\%$ on the free CPU tier and within $\pm 10\%$ on the T4 GPU runtime.

Phase	Pre-registered	Executed	Wall	Reason for scaling
1 (pilot)	$5 \cdot 10^4 \times 3 \times 4$	matches pre-reg.	~ 77 min	none.
2 (E1 sweep)	$2 \cdot 10^5 \times 5 \times 4 \times 3 \times 2$	$3 \cdot 10^4 \times 3 \times 4 \times 2 \times 2$	<i>est.</i> ~ 3 h	CPU envelope.
3 (MPE)	$\geq 10^6 \times 5 \times 4 \times 2$	deferred	–	CPU envelope; adapter shipped.
4 (ablation)	$\geq 5 \cdot 10^4 \times \dots$	$2 \cdot 10^4 \times 4 \times 3 \times 3$	<i>est.</i> ~ 2 h	CPU envelope.